

Assembly and Assemblers

COMS10015

Dr Tom Deakin

University of Bristol

Week 22

Assembly programs

Assembly

- ▶ The Instruction Set Architecture (ISA) is the interface between software and hardware.
- ▶ Gives a set of commands that manipulate the state of a processor.
- ▶ Writing programs an ISA is known as programming in *assembly language*.
- ▶ Each assembly program targets just one ISA.
- ▶ Assembly is more like a style of programming, rather than one specific language.
- ▶ Need to know details about the particular processor: number of registers, list of instructions, addressing modes, ...

Assembly programming

- ▶ Form of procedural programming.
- ▶ Each statement is executed in turn in the order listed in the program.
- ▶ Programmers use **mnemonics** to refer to each instruction.
- ▶ In Hex 8 these are the words LDAC, BRN, ...
- ▶ Contrast to *machine code* which is the binary equivalent: 0011, 1011, ...
- ▶ **Much** easier to read/write programs in assembly than machine code.

Margaret Hamilton

Director, Software Engineering Division, MIT Instrumentation Laboratory



Picture from Wikipedia

- ▶ Paper binders full of source code for Apollo 11 lunar module and command module.
- ▶ Written **in assembly** by Margaret Hamilton (age 33) and her team.
- ▶ Code available on GitHub:
<https://github.com/chrislgarry/Apollo-11>

Assembly today

- ▶ Nowadays we usually just program using high-level programming languages and facilities provided by the operating system.
- ▶ Understanding assembly very useful skill.
- ▶ Helps see what a compiler is doing — is it outputting the “best” code?
- ▶ May need to write assembly to specialise program to get best performance for critical loops on particular architecture.
- ▶ Used for low-level micro-benchmarking of processors.

Assemblers

Assembler (1)

- ▶ In order to run an assembly program, need to convert it to machine code.
- ▶ This is the job of the *assembler*.

Definition: Assembler

An *assembler* is a program that translates an assembly program into machine code.

Assembler (2)

- ▶ Assemblers take program files as input.
- ▶ Output machine code that can be loaded into memory and run.
- ▶ Assembler makes one (or more) passes over the input program.
- ▶ Translates each instruction into machine code.

Assembler (3)

- ▶ Assembler stores a lookup table which contains the mnemonics and the opcode.
- ▶ Also knows the structure of the assembly language.
- ▶ A *lexer* is used to turn the program source into tokens.
- ▶ A *token* is a string with a meaning.
- ▶ E.g. The input string LDAP 4 is split into an opcode LDAP and an operand 4.
- ▶ Meaning of the symbols defined by a *lexical grammar*.

Assembler (4)

Question

The input assembly program contains the following string somewhere: `PI`. What does this mean?

Assembler (4)

Question

The input assembly program contains the following string somewhere: `PI`. What does this mean? It depends! Could be named constant, variable, instruction, an error, or something else entirely.

The lexer has to know to assign it to the right token type.

Assembler (5)

- ▶ *Syntactic analysis* ensured the tokens appear in the correct order for the language.
- ▶ Helps enforce programs are written correctly.
- ▶ Similar to (modern) languages where need to form grammatically correct sentences.
- ▶ E.g. `ADD 12` would be an error because the `ADD` opcode does not require an operand.
- ▶ Output from this phase is often an internal representation of the program inside the assembler.

Compilers

- ▶ Compilers perform many of these same steps to compile high level programming languages.
- ▶ Read input files, a lexer breaks it into tokens, and syntax analysis.
- ▶ Key difference is the input and output.
- ▶ Assembler converts assembly programs to machine code.
- ▶ Compiler converts programs written in a higher level programming language to machine code.
- ▶ More on compilers later in the course.

Assembler abstractions

- ▶ To make things easier, the assembler usually allows for some convenient notations.
- ▶ Usually allow for:
 - ▶ labels: to mark lines of the program for cross-reference later.
 - ▶ named constants: `THE_ANSWER EQU 42` (in x86 and Arm).
 - ▶ variables: `foo SETA 100`, and use `$(foo)`.
- ▶ Assembler knows how to convert these abstractions into constants understood by the instructions.

Hex 8 assembly

General rules for Hex 8, which often apply to most assembly languages:

- ▶ Only one instruction mnemonic per line.
- ▶ Operands are expressed in decimals.
- ▶ Format of each line is: `mnemonic_operand`
- ▶ E.g. LDAC 9
- ▶ Assume program counter and memory space begin at address 0.

Assembler abstractions for Hex 8

- ▶ When writing programs in Hex 8 assembly, can make it easier with a few additions.
- ▶ These are all translated by the assembler as the processor doesn't understand them.
- ▶ Higher level abstraction over the ISA for programmer convenience.

Assembler abstractions for Hex 8: Labels (1)

Labels

- ▶ Labels in Hex 8 give a name to the *next* instruction.
- ▶ Can be used as operands.
- ▶ Both forward and backward referencing is allowed.
- ▶ Syntax is L1234.
- ▶ Must appear on their own line unless used as an operand.

In most ISAs, labels often of the form `label: inst operands`.

Assembler abstractions for Hex 8: Labels (2)

For example:

L1

LDAC 5

LDBC L2

STAI 0

BR L23

Assembler abstractions for Hex 8: Operands

Operands

- ▶ Operands are decimal numbers in the range $[-128 : 255]$.
- ▶ The assembler will insert PFIX instructions for:
 - ▶ Negative operands.
 - ▶ Operands larger than 15.
- ▶ Operands can be labels.

Because the opcode is only 4-bits, the assembler must insert PFIX instructions. My assembler is pessimistic and always inserts them, but a better one could remove the ones that aren't used.

Assembler abstractions for Hex 8: Data statements

Data

- ▶ The assembler understands DATA statements.
- ▶ This represents that the operand should be inserted into the instruction stream.
- ▶ Labels can refer to DATA statements.

These could be used to give space for global program variables, data, constants, ...

Hex 8 Assembly program

```
1:    BR L1           ; jump to line 5, L1 -> 5
2:    L2              ; label line 3 as L2 -> 3
3:    DATA 0         ; mem[3] set to 0
4:    L1              ; label line 5 as L1 -> 5
5:    LDAC 1          ; set areg to 1
6:    LDBC 2          ; set breg to 2
7:    ADD             ; set areg to 1+2 = 3
8:    LDBC 3          ; set breg to 3
9:    ADD             ; set areg to 3+3 = 6
10:   STAM L2         ; store areg into mem[L2] (mem[3])
11:   BR -2           ; jump to mem[254] (!)
```

Label resolution

Resolving labels

- ▶ Assembly programs use labels to refer to specific instructions.
- ▶ Makes writing jumps/branches easier.
- ▶ No need to know exactly how much to update the program counter by.
- ▶ But assembler needs to resolve those labels.
- ▶ Replace each *use* of the label into where it references in the output machine code.

The challenge

- ▶ Allowed to forward reference (use) labels that haven't been defined yet.
- ▶ Recall: program stack for subroutine calls.
- ▶ Might use a label for location of subroutine: `BR my_function`
- ▶ Set `mem[0]` to jump instruction to start of program: `BR START`

As the assembler reads the input one line at a time, doesn't know the target of the labels when they are used.

Two-pass assembler (1)

1.
 - ▶ Read input file to tokens.
 - ▶ Check for label, and store label in symbol table with memory location of labelled instruction.
 - ▶ Keeps running total of output instruction sizes and builds symbol table.
2.
 - ▶ Read input file to tokens.
 - ▶ Output binary representation of instructions (machine code).
 - ▶ Symbol table used to resolve labels.

Two-pass assembler (2)

- ▶ Input program is read twice.
- ▶ First pass just used to learn about labels and other symbols.
- ▶ Machine code output on second pass.
- ▶ Do not need to hold entire program in memory, just the symbol table and counters to keep track of output instruction stream.

Backpatching (1)

- ▶ One approach is to resolve labels with one pass of the input code.
- ▶ Known as *backpatching*.
- ▶ Instructions translated directly to machine code.
- ▶ If label already seen, then can calculate the location/offset and output the translated instruction.
- ▶ If label not yet seen, store instruction and label in symbol table.
- ▶ When that label is encountered, consult table and resolve those instructions.

Backpatching (2)

- ▶ Requires whole output instruction stream to be held in memory.
- ▶ Not a problem for general modern processors, but may be for embedded devices.
- ▶ Was a convenient approach when programs were stored on tapes/punch cards and processor had limited memory.
- ▶ Alternatively, could provide symbol table and instructions to linker/loader.

Further Reading

- ▶ Assemblers: <http://homepage.divms.uiowa.edu/~jones/syssoft/notes/>
- ▶ https://en.wikipedia.org/wiki/Lexical_analysis
- ▶ Assemblers, Linkers, and the SPIM Simulator:
http://pages.cs.wisc.edu/~larus/HP_AppA.pdf
- ▶ MIPS assembly parser:
<http://dmitrysoshnikov.com/compilers/mips-assembly-parser/>
- ▶ Transputer Instruction Set: a compiler writer's guide:
<http://www.transputer.net/iset/pdf/tis-acwg.pdf>
- ▶ https://en.wikipedia.org/wiki/Assembly_language
- ▶ Arm Assembly Language: <https://developer.arm.com/documentation/dui0473/c/writing-arm-assembly-language>